SWEET1NE RESTAURANT MANAGEMENT SYSTEM

Dynamic Role & Permission Engine Design

Internal Developer Architecture Document

Prepared by	Stephen Nyansoho Machera
Organization	SpectreTech Limited
Prepared for	Sweet1ne Restaurant
Date	1 August 2026

1 Purpose

The objective is to build a fully configurable Role-Based Access Control (RBAC) system that allows the restaurant's Super Admin to manage staff access without requiring developer intervention whenever responsibilities change.

The system should support future operational growth while keeping the codebase clean, maintainable, and secure. The RBAC engine must be generic and reusable across all modules of the application.

2 Design Philosophy

The system must never assign permissions directly to users. Instead it follows this hierarchy:

```
Permissions
    ↓
Roles
    ↓
Users
```

Permissions define individual capabilities. Roles are collections of permissions. Users receive one assigned role.

This keeps permission management centralized, reduces administrative overhead, and prevents inconsistent access configurations.

3 Permission Registry

Permissions represent atomic actions within the application.

```
orders.view       orders.accept       orders.complete       orders.cancel
menu.view         menu.create          menu.edit             menu.delete
analytics.view    analytics.export
staff.view        staff.create         staff.edit            staff.delete
campaigns.view    campaigns.send
settings.view     settings.edit
roles.manage      users.manage
```

Permission keys are defined by developers. The Super Admin cannot create new permission keys.

Reason: permissions correspond directly to application functionality implemented in code. A permission with no corresponding feature has no meaning. Developers extend the registry whenever new functionality is introduced.

4 Dynamic Roles

Roles are completely configurable. The Super Admin can create unlimited roles.

- Chef
- Kitchen Supervisor
- Floor Staff
- Branch Manager
- Marketing
- Reception
- Technical Support

Each role contains a custom combination of permissions selected from the permission registry.

Role	Allowed	Not allowed
Chef	orders.view; orders.complete; menu.view	menu.edit; analytics.view; staff.manage
Branch Manager	orders.view; menu.edit; analytics.view; staff.view; campaigns.view	roles.manage
Marketing	campaigns.send; analytics.view	orders.complete; menu.edit

No code changes are required when creating new roles.

5 User Assignment

Users receive exactly one role.

```
John  →  Chef
Sarah →  Branch Manager
```

Permissions are inherited entirely from the assigned role. The system must not support per-user permissions. This avoids complexity and keeps permission management predictable.

6 Branch Scope

Sweet1ne operates multiple branches. Each staff account should also be associated with one or more branches.

```
Branch Manager  →  Chingford
Branch Manager  →  Lewisham
```

The permission system determines what a user can do. Branch scope determines where they can do it. This separation keeps the design flexible.

7 Database Design

Table	Fields
Permissions	id; key; display_name; category; description; created_at
Roles	id; name; description; created_at; created_by; active
Role Permissions	role_id; permission_id
Users	id; email; role_id; branch_id; active

No permission information should be stored on the User table.

8 Dashboard Architecture

The application should not create different dashboards for different roles. Instead, it should have one shared dashboard shell:

```
/dashboard
Authenticate → Load User → Load Assigned Role
→ Load Permissions → Load Dashboard → /dashboard
```

The shell remains identical for every user. Only the visible modules change.

9 Dashboard Shell

The shared dashboard should contain:

```
Sidebar     Header     Breadcrumbs
Notifications     User Menu     Main Content
```

These components are reused by every role. The dashboard shell should never be duplicated.

10 Permission-Driven Navigation

Sidebar items should be defined as configuration. Each navigation item contains a required permission.

Navigation	Required permission
Orders	orders.view
Analytics	analytics.view
Settings	settings.view

Role	Visible navigation
Chef	Dashboard; Orders; Profile
Branch Manager	Dashboard; Orders; Menu; Analytics; Staff
Marketing	Dashboard; Campaigns; Analytics

When permissions are loaded, the sidebar automatically filters itself. The dashboard remains identical; only navigation changes.

11 Route Protection

Hiding navigation items is not sufficient. Every page must validate permissions on the server.

Route	Required permission
/dashboard/menu	menu.view
/dashboard/settings	settings.view

Unauthorized users receive a 403 response. Security must never depend on hidden buttons.

12 Permission-Based Components

Permissions should also control individual UI actions.

Menu page action	Required permission
View Menu	menu.view
Edit Button	menu.edit
Delete Button	menu.delete

Therefore, two users can access the same page while seeing different actions.

13 Dashboard Widgets

Dashboard widgets should also use permissions. Possible widgets include:

- Today's Orders
- Revenue
- Pending Orders
- Top Selling Dish
- Campaign Performance
- Staff Online
- Reservations

Role	Widgets
Chef	Today's Orders; Pending Orders
Branch Manager	Revenue; Orders; Staff; Top Selling Dish
Marketing	Campaign Performance; Customer Growth; Email Opens

Widgets should automatically render based on permissions.

14 Super Admin Rules

The system should contain one protected Super Admin role.

Restrictions:

- Cannot be deleted.
- Cannot lose all permissions.
- The final Super Admin account cannot be disabled.
- Cannot accidentally lock the system.

The Super Admin manages:

- Roles
- Permissions
- Users
- Branch assignments

15 Role Deletion Rules

Roles currently assigned to users cannot be deleted. Instead display:

```
“This role is assigned to X users. Reassign those users before deleting the role.”
```

This prevents accidental loss of access.

16 shadcn/ui Integration

The UI should be built using shadcn/ui components to ensure consistency, accessibility, and maintainability.

Area	Recommended components / use
Layout	Sidebar; Sheet (mobile navigation); Breadcrumb; Separator
Dashboard	Card; Badge; Avatar; Scroll Area
Forms	Input; Label; Select; Checkbox; Switch; Radio Group; Textarea; Calendar; Combobox
Dialogs	Dialog; Alert Dialog; Drawer - Create Role, Edit Role, Assign User, Delete Confirmation
Data Display	Table; Data Table (TanStack Table); Accordion; Collapsible; Tabs
Notifications	Toast; Alert; Progress
Search	Command Component - global, staff, role, and permission search

17 Role Management UI

The Role Management page should provide:

- Create Role
- Edit Role
- Delete Role
- Duplicate Role
- Search Roles
- Enable/Disable Role
- Assign Permissions
- View Assigned Users

Permission selection should be grouped into categories.

Category	Selected	Not selected
Orders	View; Accept	Cancel
Menu	View; Edit	Delete
Analytics	View	Export
Staff	View	Create; Delete

This interface should use Accordion, Checkbox, Badge, Search, and Dialog components provided by shadcn/ui.

18 Development Principles

- One dashboard shell for all users.
- Permissions drive navigation.
- Permissions drive widgets.
- Permissions drive page access.
- Permissions drive button visibility.
- Roles are dynamic.
- Permission keys are fixed and developer-managed.
- Users receive one role only.
- Server-side authorization is mandatory.
- UI visibility must never be treated as security.

19 Expected Outcome

The completed RBAC system should allow the restaurant to evolve its organizational structure without requiring software changes.

New staff roles can be introduced through the administration interface. Existing roles can be modified without redeployment.

Every user receives a personalized dashboard assembled dynamically from a shared dashboard shell based on their assigned permissions.

This architecture minimizes duplicated code, simplifies long-term maintenance, and provides a scalable foundation for future modules while preserving strong server-side security.